

Handoff Comercial SMB → Customer Success/KAM

Documentación técnica-funcional del flujo de datos no cualitativos

Destinataria: Tatiana — Process Automation Developer

Implementación objetivo: B-maker / Next.js

Producto de consulta: KAM360

Versión: 1.0 — 26 de agosto de 2026

Estado: Documento base para validación técnica

1. Resumen ejecutivo

KAM360 es la capa de consulta del proceso de Handoff Comercial SMB → Customer Success/KAM. Su propósito es reunir, por comercio, dos fuentes de información:

1. Datos no cualitativos, consultados desde fuentes internas: identificación, clasificación, estructura comercial, TPV, actividad, productos y terminales.
2. Contexto cualitativo, capturado en el formulario de handoff: contactos adicionales, historia del comercio, modelo de atención, negociaciones, necesidades, potencial y compromisos.

El principio de diseño es simple:

Si el dato ya existe o puede calcularse de forma confiable en las fuentes internas, no debe pedirse manualmente en el formulario.

merchant_id es la llave canónica de integración. La salida recomendada es una vista o tabla curada, BASE_COMERCIOS, que B-maker/Next.js pueda consumir sin reproducir lógica analítica compleja en la interfaz.

Decisión recomendada

La consulta a Athena/Metabase debe ejecutarse en una capa de backend o automatización controlada. El navegador no debe conectarse directamente al catálogo ni contener credenciales. La aplicación debe consumir un contrato estable mediante API, vista materializada o tabla sincronizada.

2. Objetivo y alcance

2.1 Objetivo

Documentar cómo obtener, transformar, validar y publicar los datos no cualitativos que enriquecen el handoff, para que Tatiana pueda implementar la visualización en B-maker con Next.js y el equipo pueda mantener el flujo de manera segura y escalable.

2.2 Incluye

- arquitectura lógica y responsabilidades por capa;
- contrato esperado de BASE_COMERCIOS;

- fuentes/marts candidatos;
- consultas SQL base para descubrimiento y construcción;
- reglas de transformación y deduplicación;
- validaciones de calidad y reconciliación;
- integración recomendada con B-maker/Next.js;
- separación entre datos automáticos y campos del formulario;
- criterios de automatización, seguridad, observabilidad y escalabilidad.

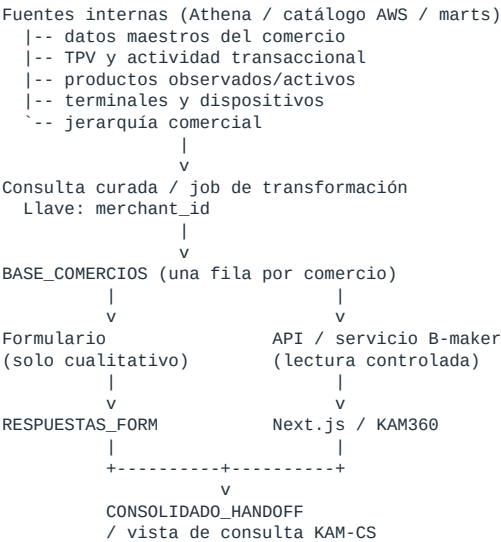
2.3 No incluye

- credenciales, secretos o configuración de acceso;
- definición definitiva de columnas que aún no se ha validado en el catálogo;
- lógica de asignación de KAM, salvo que el negocio entregue una fuente oficial;
- información cualitativa que depende del conocimiento del Team Lead.

Nota de implementación: los nombres de tablas y columnas incluidos son candidatos basados en el diseño actual. Antes de pasar a producción deben verificarse en `information_schema`, Metabase/Athena y con los dueños de datos.

3. Arquitectura propuesta

TEXT



3.1 Responsabilidades por capa

Capa	Responsabilidad	No debe hacer
Fuentes/marts	Conservar hechos y dimensiones oficiales	Adaptarse a la UI
Transformación	Unificar, deduplicar, calcular métricas y publicar el contrato	Exponer secretos al cliente
BASE_COMERCIOS	Entregar una fila estable por merchant_id	Guardar respuestas cualitativas como fuente maestra
Formulario	Capturar conocimiento humano no disponible en bases	Pedir TPV, productos o terminales
API/B-maker	Autorizar, filtrar y servir datos	Ejecutar SQL arbitrario enviado por el navegador
Next.js/KAM360	Buscar, filtrar y presentar el handoff	Reimplementar reglas analíticas complejas

4. Llave, granularidad y reglas del contrato

- Llave primaria lógica: merchant_id.
- Granularidad de BASE_COMERCIOS: una fila por comercio.
- Granularidad de terminales: una fila por terminal en su fuente; se agrega antes del cruce.
- Granularidad de TPV: una fila por comercio y día, o por transacción; se agrega antes del cruce.
- Granularidad del formulario: puede haber varias respuestas por comercio; para la vista operativa se toma la respuesta válida más reciente, conservando el histórico.
- Zona horaria funcional: America/Bogota, salvo que la plataforma defina UTC como estándar. La API debe devolver timestamps ISO 8601.
- Nulos: un valor desconocido no debe convertirse automáticamente en cero. Cero significa medición válida con resultado cero; nulo significa ausencia o dato no disponible.

5. Estructura esperada de BASE_COMERCIOS

5.1 Contrato mínimo — requerido para el MVP

Campo	Tipo lógico	Regla / descripción
merchant_id	string	Obligatorio, único, sin espacios laterales; no convertir a número
merchant_name	string	Nombre visible del comercio
document_type	string	Tipo de documento homologado
document_number	string	Mantener como texto; proteger ceros a la izquierda
category_id	string	Categoría del comercio
subcategory_id	string	Subcategoría del comercio
city_code	string	Código oficial disponible en la fuente
address	string	Dirección principal disponible
email	string	Correo principal del comercio; dato personal restringido
cellphone_number	string	Teléfono principal; dato personal restringido
sales_agent_email	string	Ejecutivo comercial responsable
tpv_max	decimal	Máximo TPV según ventana acordada
clasificacion	string	Clasificación/segmento homologado, sin tilde en nombre técnico
team_lead	string	Team Lead vigente según fuente comercial
manager	string	Manager vigente según fuente comercial
source_updated_at	timestamp	Última actualización efectiva de las fuentes
record_updated_at	timestamp	Momento en que se generó el registro curado

5.2 Contrato enriquecido — recomendado

Campo	Tipo lógico	Definición sugerida
city	string	Nombre homologado de ciudad
onboarding_end_date	date	Fecha oficial de fin de onboarding/activación
first_transaction_date	date	Primera transacción válida
last_transaction_date	date	Última transacción válida
tpv_30d	decimal	TPV de los últimos 30 días completos o ventana acordada
tpv_90d	decimal	TPV de los últimos 90 días completos
tpv_avg_3m	decimal	Promedio mensual de TPV en 3 meses completos
products_active	array/string	Lista normalizada de productos activos/observados
numero_terminales	integer	Conteo de terminales según definición acordada
terminales_asociadas	array/string	IDs/seriales autorizados para mostrar
ultima_actualizacion_terminal	timestamp	Último cambio observado en terminales
tiene_pos	boolean	Regla oficial de actividad/disponibilidad POS
tiene_link_pago	boolean	Regla oficial de Link de pago
tiene_boton_pago	boolean	Regla oficial de Botón de pago
tiene_qr	boolean	Regla oficial de QR
tiene_cuenta_bold	boolean	Regla oficial de Cuenta Bold
tiene_bolsillo	boolean	Regla oficial de Bolsillo
data_quality_status	string	OK, WARNING o ERROR

5.3 Ejemplo de respuesta API

JSON

```
{
  "merchant_id": "mrc_12345",
  "merchant_name": "Comercio ejemplo",
  "city": "Bogota",
  "clasificacion": "SMB",
  "team_lead": "Nombre TL",
  "manager": "Nombre manager",
  "tpv_30d": 125000000.00,
  "tpv_avg_3m": 119500000.00,
  "numero_terminales": 4,
  "terminales_asociadas": ["TERM-001", "TERM-002"],
  "products_active": ["POS", "LINK_PAGO"],
  "source_updated_at": "2026-08-26T08:00:00-05:00",
  "data_quality_status": "OK"
}
```

6. Fuentes y marts sugeridos

Fuente candidata	Uso esperado	Validación previa
awsdatacatalog.bold_gold_growth.dim_client	Maestro del comercio, documento, contacto, ciudad y fechas	Confirmar llave, vigencia y columna de actualización
awsdatacatalog.bold_gold_growth.mart_tpv_daily_by_merchant	TPV diario agregado por comercio	Confirmar estados y tipos de transacción incluidos
awsdatacatalog.bold_gold_growth.mart_tpv_daily_by_transaction	Actividad/productos por tipo transaccional	Confirmar taxonomía y evitar doble conteo
awsdatacatalog.bold_gold_growth.dim_terminal	Atributos actuales de terminal/dispositivo	Confirmar si contiene merchant_id y estado vigente
awsdatacatalog.bold_gold_growth.fact_terminal_history	Historial de asociación y estados	Definir registro vigente por fecha efectiva
awsdatacatalog.bold_gold_growth.mart_terminal_enrich	Relación enriquecida comercio-terminal	Confirmar granularidad y cobertura
awsdatacatalog.bold_gold_growth.mart_master_merchant_enrich	Vista consolidada del comercio	Evaluar si puede ser fuente principal
awsdatacatalog.bold_gold_growth.mart_master_merchant_lineage	Linaje/homologación de merchant	Confirmar uso para merges o cambios de ID
Fuente comercial oficial	Ejecutivo, Team Lead, manager y clasificación	Definir vigencia y dueño del dato

6.1 Matriz de decisión de fuente

Para cada campo se debe registrar: fuente oficial, columna, definición, granularidad, frecuencia, dueño y regla de desempate. Si dos fuentes difieren, no se debe elegir por conveniencia técnica: se debe acordar cuál es autoritativa.

7. Consultas SQL base

Las siguientes consultas usan sintaxis compatible con Athena/Trino. Deben adaptarse a los nombres reales encontrados en el catálogo.

7.1 Descubrir columnas relevantes

SQL

```
SELECT
    table_schema,
    table_name,
    column_name,
    data_type
FROM information_schema.columns
WHERE regexp_like(
    lower(column_name),

    'merchant_id|merchant_name|document|city|terminal|device|serial|pos|product|tpv|onboarding|activation|up
dated'
)
ORDER BY table_schema, table_name, column_name;
```

7.2 Encontrar tablas con merchant_id y datos de terminal

SQL

```
WITH candidate_columns AS (
    SELECT
        table_schema,
        table_name,
        lower(column_name) AS column_name
    FROM information_schema.columns
    WHERE regexp_like(
        lower(column_name),
        'merchant_id|terminal|dataphone|datafono|device|serial|pos|reader'
    )
)
SELECT
    table_schema,
    table_name,
    array_join(array_agg(column_name ORDER BY column_name), ', ') AS columns_found
FROM candidate_columns
GROUP BY table_schema, table_name
HAVING count_if(column_name = 'merchant_id') > 0
    AND count_if(regexp_like(
        column_name,
        'terminal|dataphone|datafono|device|serial|pos|reader'
    )) > 0
ORDER BY table_schema, table_name;
```

7.3 Maestro de comercios — patrón de deduplicación

SQL

```
WITH client_ranked AS (
    SELECT
        cast(merchant_id AS varchar) AS merchant_id,
        merchant_name,
        merchant_identification_document_type AS document_type,
        cast(merchant_identification_document_number AS varchar) AS document_number,
        category_id,
        subcategory_id,
        city_code,
        city,
        address,
        email,
        cellphone_number,
        onboarding_end_date,
        updated_at,
        row_number() OVER (
            PARTITION BY merchant_id
            ORDER BY updated_at DESC
        ) AS rn
    FROM awsdatacatalog.bold_gold_growth.dim_client
    WHERE merchant_id IS NOT NULL
)
SELECT *
FROM client_ranked
WHERE rn = 1;
```

Reemplazar updated_at y los nombres de identificación por las columnas confirmadas. Si la dimensión es SCD, usar sus fechas de vigencia o bandera is_current en lugar de inferir vigencia únicamente con updated_at.

7.4 TPV por ventanas completas

SQL

```
WITH params AS (
    SELECT current_date AS run_date
),
tpv AS (
    SELECT
        cast(t.merchant_id AS varchar) AS merchant_id,
        min(CASE WHEN t.tpv > 0 THEN t.transaction_date END)
            AS first_transaction_date,
        max(CASE WHEN t.tpv > 0 THEN t.transaction_date END)
            AS last_transaction_date,
        sum(CASE
            WHEN t.transaction_date >= date_add('day', -30, p.run_date)
            THEN t.tpv ELSE 0 END
        ) AS tpv_30d,
        sum(CASE
```

```

        WHEN t.transaction_date >= date_add('day', -90, p.run_date)
        THEN t.tpv ELSE 0 END
    ) AS tpv_90d,
    max(t.tpv) AS tpv_max_daily
FROM awsdatalog.bold_gold_growth.mart_tpv_daily_by_merchant t
CROSS JOIN params p
WHERE t.transaction_date >= date_add('day', -365, p.run_date)
    -- Agregar filtros oficiales de estado, moneda y reversos.
GROUP BY cast(t.merchant_id AS varchar)
)
SELECT
    merchant_id,
    first_transaction_date,
    last_transaction_date,
    tpv_30d,
    tpv_90d,
    tpv_90d / 3.0 AS tpv_avg_3m,
    tpv_max_daily
FROM tpv;

```

El negocio debe acordar si “30 días” significa ventana móvil o último mes calendario cerrado. Para comparabilidad operativa se recomienda usar meses cerrados y publicar la fecha de corte.

7.5 Terminales asociadas

SQL

```

WITH terminal_ranked AS (
    SELECT
        cast(merchant_id AS varchar) AS merchant_id,
        cast(terminal_id AS varchar) AS terminal_id,
        cast(serial_number AS varchar) AS serial_number,
        terminal_status,
        terminal_model,
        updated_at,
        row_number() OVER (
            PARTITION BY merchant_id, terminal_id
            ORDER BY updated_at DESC
        ) AS rn
    FROM awsdatalog.bold_gold_growth.mart_terminal_enrich
    WHERE merchant_id IS NOT NULL
),
terminal_current AS (
    SELECT *
    FROM terminal_ranked
    WHERE rn = 1
        AND upper(coalesce(terminal_status, 'UNKNOWN')) NOT IN
            ('DELETED', 'RETURNED', 'CANCELLED')
)
SELECT
    merchant_id,
    count(DISTINCT terminal_id) AS numero_terminales,
    array_sort(array_distinct(array_agg(terminal_id))) AS terminales_asociadas,
    max(updated_at) AS ultima_actualizacion_terminal
FROM terminal_current
GROUP BY merchant_id;

```

La definición de “terminal asociada” debe acordar si incluye inactivas, en inventario, devueltas o sin transacciones recientes. No mostrar seriales si la política de seguridad solo autoriza IDs internos.

7.6 Productos activos u observados

SQL

```

WITH product_activity AS (
    SELECT
        cast(merchant_id AS varchar) AS merchant_id,
        upper(trim(product_name)) AS product_name,
        max(transaction_date) AS last_activity_date
    FROM awsdatalog.bold_gold_growth.mart_tpv_daily_by_transaction
    WHERE transaction_date >= date_add('day', -90, current_date)
    -- Agregar filtros oficiales de transacción aprobada/no reversada.
    GROUP BY cast(merchant_id AS varchar), upper(trim(product_name))
)
SELECT
    merchant_id,
    max(CASE WHEN regexp_like(product_name, 'POS') THEN true ELSE false END)
        AS tiene_pos,
    max(CASE WHEN regexp_like(product_name, 'LINK') THEN true ELSE false END)

```



```

        AS tiene_link_pago,
        max(CASE WHEN regexp_like(product_name, 'BOTON|BOTÓN') THEN true ELSE false END)
        AS tiene_boton_pago,
        max(CASE WHEN regexp_like(product_name, 'QR') THEN true ELSE false END)
        AS tiene_qr,
        array_sort(array_distinct(array_agg(product_name))) AS products_active
FROM product_activity
GROUP BY merchant_id;

```

Una transacción reciente prueba actividad observada, no necesariamente habilitación contractual. Si existe una fuente oficial de productos habilitados, debe prevalecer y la señal transaccional puede mantenerse como atributo separado (product_observed_90d).

7.7 Query integrada para BASE_COMERCIOS

SQL

```

WITH
clients AS (
  -- Sustituir por la consulta validada de la sección 7.3.
  SELECT * FROM curated.client_current
),
tpv AS (
  -- Sustituir por la consulta validada de la sección 7.4.
  SELECT * FROM curated.tpv_by_merchant
),
terminals AS (
  -- Sustituir por la consulta validada de la sección 7.5.
  SELECT * FROM curated.terminals_by_merchant
),
products AS (
  -- Sustituir por la consulta validada de la sección 7.6.
  SELECT * FROM curated.products_by_merchant
),
commercial AS (
  SELECT
    cast(merchant_id AS varchar) AS merchant_id,
    sales_agent_email,
    clasificacion,
    team_lead,
    manager
  FROM curated.commercial_assignment_current
)
SELECT
  c.merchant_id,
  c.merchant_name,
  c.document_type,
  c.document_number,
  c.category_id,
  c.subcategory_id,
  c.city_code,
  c.city,
  c.address,
  c.email,
  c.cellphone_number,
  a.sales_agent_email,
  a.clasificacion,
  a.team_lead,
  a.manager,
  c.onboarding_end_date,
  t.first_transaction_date,
  t.last_transaction_date,
  t.tpv_30d,
  t.tpv_90d,
  t.tpv_avg_3m,
  t.tpv_max_daily AS tpv_max,
  coalesce(te.numero_terminales, 0) AS numero_terminales,
  te.terminales_asociadas,
  te.ultima_actualizacion_terminal,
  coalesce(p.tiene_pos, false) AS tiene_pos,
  coalesce(p.tiene_link_pago, false) AS tiene_link_pago,
  coalesce(p.tiene_boton_pago, false) AS tiene_boton_pago,
  coalesce(p.tiene_qr, false) AS tiene_qr,
  p.products_active,
  greatest(
    c.updated_at,
    te.ultima_actualizacion_terminal,
    t.last_transaction_date
  ) AS source_updated_at,
  current_timestamp AS record_updated_at
FROM clients c

```

```

LEFT JOIN commercial a ON c.merchant_id = a.merchant_id
LEFT JOIN tpv t ON c.merchant_id = t.merchant_id
LEFT JOIN terminals te ON c.merchant_id = te.merchant_id
LEFT JOIN products p ON c.merchant_id = p.merchant_id;

```

7.8 Consulta segura por lista de comercios

Si el alcance inicial es la lista del handoff, cargarla como tabla controlada y cruzarla:

SQL

```

SELECT b.*
FROM curated.base_comercios b
INNER JOIN control.handoff_scope h
    ON b.merchant_id = h.merchant_id
WHERE h.batch_id = :batch_id;

```

No concatenar listas ni valores recibidos desde la URL dentro del SQL. Usar parámetros o una tabla de alcance administrada.

8. Validaciones de datos

8.1 Controles obligatorios antes de publicar

Control	Regla de aceptación	Acción si falla
Unicidad	0 duplicados por merchant_id	Bloquear publicación
Llave nula	0 merchant_id nulos o vacíos	Bloquear publicación
Cobertura	100% de comercios del lote aparecen en la salida	Alertar y conciliar faltantes
Cardinalidad	Cada join agregado conserva una fila por comercio	Bloquear si multiplica filas
Frescura	Fuentes dentro del SLA acordado	Marcar WARNING y mostrar corte
TPV	Sin negativos no explicados; conciliación con total oficial	Bloquear o excluir según regla
Terminales	Conteo coherente con estado y fuente oficial	Revisar muestra y regla de vigencia
Jerarquía comercial	TL/manager dentro de catálogo vigente	Alertar al dueño comercial
PII	Solo campos autorizados; acceso por rol	Bloquear exposición

8.2 SQL de controles

SQL

```

-- Unicidad y llaves nulas
SELECT
    count(*) AS row_count,
    count(DISTINCT merchant_id) AS distinct_merchants,
    count_if(merchant_id IS NULL OR trim(merchant_id) = '') AS null_keys
FROM curated.base_comercios;

-- Duplicados
SELECT merchant_id, count(*) AS rows_per_merchant
FROM curated.base_comercios
GROUP BY merchant_id
HAVING count(*) > 1;

```

```
-- Cobertura del lote
SELECT h.merchant_id
FROM control.handoff_scope h
LEFT JOIN curated.base_comercios b
  ON h.merchant_id = b.merchant_id
WHERE h.batch_id = :batch_id
  AND b.merchant_id IS NULL;

-- Frescura
SELECT
  max(record_updated_at) AS last_refresh,
  date_diff('hour', max(record_updated_at), current_timestamp) AS age_hours
FROM curated.base_comercios;
```

8.3 Reconciliación mínima

Antes del go-live, comparar una muestra de comercios contra Metabase o la fuente oficial:

- 5 comercios con TPV alto;
- 5 con TPV cero o sin actividad;
- 5 con varias terminales;
- 5 con cambio reciente de Team Lead/manager;
- 5 con productos variados.

Registrar resultado, diferencia, causa, responsable y decisión. La aprobación funcional debe involucrar al dueño del proceso comercial y al dueño de los datos.

9. Campos que NO deben pedirse en el formulario

Siempre que la fuente esté validada y fresca, estos campos se obtienen automáticamente:

- merchant_id escrito manualmente; debe derivarse de la selección del comercio;
- nombre, documento, categoría, subcategoría, ciudad, dirección, email y teléfono principal;
- ejecutivo, Team Lead, manager y clasificación;
- TPV histórico, máximo, promedio y ventanas recientes;
- fechas de onboarding y primera/última transacción;
- productos activos u observados: POS, Link, Botón, QR, Cuenta Bold y Bolsillo;
- cantidad, IDs y estado de terminales/datáfonos;
- comodato y fechas asociadas, si existe una fuente contractual confiable;
- prioridad calculable y KAM asignado, cuando existan reglas/fuentes oficiales;
- correo del respondiente como pregunta manual; si se requiere trazabilidad, capturarlo mediante identidad autenticada.

9.1 Campos que sí permanecen en el formulario

- selección del comercio;
- página web, si se desea validar o actualizar;
- contactos adicionales: tipo, nombre, cargo, teléfono, email, mejor hora y vía;
- grupo empresarial, múltiples sedes y alcance nacional cuando no exista fuente oficial;
- historia del comercio;
- modelo y frecuencia de atención;
- reportería especial;
- negociaciones y términos especiales;
- dolores y necesidades;
- planes de nuevas sedes;

- TPV potencial/opción de profundización;
- compromisos pendientes del cliente y de Bold.

10. Flujo operativo para B-maker / Next.js

10.1 Lectura recomendada

1. Un job programado crea o actualiza BASE_COMERCIOS.
2. El job ejecuta validaciones y publica solo si pasa los controles bloqueantes.
3. B-maker expone endpoints autenticados sobre la vista curada.
4. Next.js usa Server Components, Route Handlers o una capa server-side para consultar la API.
5. El navegador recibe únicamente los campos autorizados para el usuario.
6. La UI muestra fecha de corte y estado de calidad.

10.2 Endpoints sugeridos

TEXT

```
GET /api/merchants?query=&teamLead=&manager=&classification=&cursor=
GET /api/merchants/{merchantId}
GET /api/merchants/{merchantId}/handoff
GET /api/handoff/batches/{batchId}/progress
POST /api/handoff/responses
```

10.3 Comportamientos de la interfaz

- búsqueda por nombre o merchant_id, con debounce y paginación;
- filtros por Team Lead, manager, ciudad, clasificación y estado del handoff;
- ficha 360 con datos automáticos claramente separados del contexto declarado;
- fecha de corte visible en métricas transaccionales;
- estados de carga, vacío, error y dato desactualizado;
- no mostrar PII completa en listados; reservarla para detalle y usuarios autorizados;
- distinguir 0, No, Sin dato y No aplica;
- permitir ver el histórico de respuestas sin sobrescribirlo.

10.4 Ejemplo de Route Handler en Next.js

TS

```
// app/api/merchants/[merchantId]/route.ts
import { NextRequest, NextResponse } from "next/server";

export async function GET(
  request: NextRequest,
  { params }: { params: Promise<{ merchantId: string }> }
) {
  const { merchantId } = await params;
  const session = await requireAuthorizedSession(request);

  const response = await fetch(
    `${process.env.BMAKER_API_URL}/merchants/${encodeURIComponent(merchantId)}`,
    {
      headers: {
        Authorization: `Bearer ${session.serviceToken}`,
        "X-Correlation-Id": crypto.randomUUID(),
      },
      cache: "no-store"
    }
  );

  if (!response.ok) {
    return NextResponse.json(
      { error: "No fue posible consultar el comercio" },
    );
  }
}
```

```
    { status: response.status }  
  );  
}  
  
return NextResponse.json(await response.json());  
}
```

Los secretos deben residir en variables de entorno del servidor o en el gestor de secretos de la plataforma. Nunca usar variables `NEXT_PUBLIC_*` para credenciales.

10.5 Escritura del formulario

- validar el `merchant_id` contra el alcance activo;
- almacenar `response_id`, `merchant_id`, `batch_id`, `submitted_at`, identidad autenticada cuando corresponda, versión del formulario y payload;
- mantener respuestas `append-only`; una corrección crea una nueva versión o evento;
- usar idempotency key para evitar duplicados por reintento;
- sanitizar texto libre y limitar tamaño;
- no confiar en Team Lead/manager enviados por el cliente: resolverlos en backend desde `BASE_COMERCIOS`.

11. Criterios de automatización

11.1 Frecuencia

- maestro y jerarquía comercial: diaria o según SLA de la fuente;
- TPV: diaria después del cierre/estabilización;
- terminales/productos: diaria, o más frecuente si el caso lo exige;
- respuestas del handoff: near-real-time o cada pocos minutos;
- tablero de avance: después de cada respuesta o por actualización incremental.

11.2 Requisitos del job

- ejecución idempotente;
- partición por fecha de corte/lote;
- reintentos con backoff;
- timeout y límites de costo;
- watermark de la última carga exitosa;
- logs estructurados con `run_id`, filas leídas/escritas, duración y errores;
- alerta cuando falla, se demora o disminuye la cobertura;
- publicación atómica: construir versión nueva y activar solo después de validar;
- posibilidad de reprocesar un lote histórico sin sobrescribir evidencia.

11.3 Criterios de aceptación para el MVP

- 100% de los comercios del lote consultables;
- 0 duplicados por `merchant_id` en `BASE_COMERCIOS`;
- filtros y búsqueda responden con paginación;
- formulario no pide datos ya disponibles en fuentes;
- consolidado selecciona la respuesta válida más reciente y conserva histórico;
- fecha de corte y estado de frescura visibles;
- acceso restringido y auditado;

- error de fuente no produce una pantalla silenciosamente vacía.

12. Seguridad, privacidad y gobierno

- aplicar autenticación corporativa y autorización por roles;
- principio de mínimo privilegio para Athena, API y almacenamiento;
- cifrado en tránsito y reposo;
- secretos solo del lado servidor;
- registro de accesos a datos sensibles;
- enmascarar email, teléfono, documento y seriales cuando el rol no requiera detalle;
- definir retención para contactos y respuestas cualitativas;
- evitar PII en logs, URLs, métricas, mensajes de error o herramientas de analítica;
- documentar dueño de cada campo y canal para correcciones;
- revisar con Seguridad/Privacidad antes de ampliar audiencia o fuentes.

13. Recomendaciones de escalabilidad

Corto plazo — estabilizar

- formalizar diccionario y contrato de BASE_COMERCIOS;
- convertir las queries exploratorias en vistas curadas versionadas;
- definir fuente oficial de jerarquía comercial y productos;
- agregar pruebas de unicidad, cobertura y frescura;
- exponer API paginada y autenticada;
- separar entornos de desarrollo, pruebas y producción.

Mediano plazo — desacoplar

- reemplazar sincronizaciones manuales de Sheets por pipeline programado;
- usar almacenamiento histórico por lote/fecha de corte;
- implementar actualizaciones incrementales;
- añadir cache server-side con invalidación por versión del dataset;
- crear catálogo de métricas y lineage;
- establecer SLO de disponibilidad y frescura.

Largo plazo — operar como producto de datos

- contrato versionado (v1, v2) con compatibilidad hacia atrás;
- eventos de actualización para invalidar cache y refrescar vistas;
- reglas de calidad como código y alertas por anomalías;
- historial temporal de asignaciones y terminales;
- observabilidad de uso para priorizar campos y retirar datos sin consumo;
- ownership formal: negocio, datos, automatización y aplicación.

14. Riesgos y decisiones pendientes

Tema	Riesgo	Decisión requerida
Nombres de columnas	Las queries base pueden no coincidir con el esquema real	Validar catálogo y ajustar
Definición de TPV	Ventanas/filtros diferentes producen cifras distintas	Acordar métrica y fecha de corte
Productos	Actividad no equivale a habilitación	Definir fuente oficial y dos señales si aplica
Terminales	Estados históricos pueden inflar el conteo	Acordar qué significa "asociada/activa"
Jerarquía comercial	Cambios de TL/manager pueden quedar desactualizados	Elegir fuente vigente y frecuencia
Respuestas duplicadas	Una fila por respuesta rompe la ficha 360	Mantener histórico y elegir última válida
PII	Exposición excesiva en listados o logs	RBAC, enmascaramiento y auditoría
Dependencia de Sheets	Límites, cambios manuales y falta de contrato	Migrar a API/almacenamiento administrado

15. Plan de implementación sugerido

Fase 1 — Descubrimiento y contrato

1. Ejecutar consultas de catálogo.
2. Confirmar columnas, granularidad, estados y propietarios.
3. Aprobar diccionario y reglas de TPV/productos/terminales.
4. Crear dataset de prueba con comercios representativos.

Fase 2 — Construcción y calidad

1. Crear vistas agregadas por dominio.
2. Construir BASE_COMERCIOS con una fila por llave.
3. Automatizar controles bloqueantes y alertas.
4. Reconciliar muestra contra fuentes oficiales.

Fase 3 — B-maker / Next.js

1. Publicar endpoints autenticados y paginados.
2. Implementar búsqueda, filtros, ficha y estados de error.
3. Integrar respuestas del formulario con histórico.
4. Probar permisos, rendimiento y manejo de datos desactualizados.

Fase 4 — Puesta en producción

1. UAT con Comercial, KAM/CS, Data y Automation.
2. Documentar runbook, responsables y SLA.
3. Activar monitoreo y alertas.
4. Hacer despliegue controlado y revisión posterior.

16. Checklist de entrega a producción

- ☐ Diccionario de datos aprobado.
- ☐ Fuentes oficiales y owners confirmados.
- ☐ Definición de TPV y fecha de corte aprobadas.
- ☐ Definición de terminal activa/asociada aprobada.
- ☐ BASE_COMERCIOS tiene una fila por merchant_id.
- ☐ Cobertura del lote = 100% o excepciones documentadas.
- ☐ Queries parametrizadas; no hay SQL construido desde la URL.
- ☐ Credenciales solo en servidor/gestor de secretos.
- ☐ PII enmascarada y acceso por rol.
- ☐ Histórico de respuestas conservado.
- ☐ API con paginación, límites, timeouts y correlación.
- ☐ Fecha de corte visible en KAM360.
- ☐ Pruebas de errores, datos vacíos y fuentes desactualizadas.
- ☐ Monitoreo, alertas y runbook activos.
- ☐ UAT y aprobación funcional completadas.

17. Entregables técnicos que Tatiana debería recibir

1. Este documento y su versión PDF.
2. Diccionario final de BASE_COMERCIOS.
3. SQL validado en el entorno real, con comentarios y parámetros.
4. Resultado de las validaciones de calidad.
5. Contrato OpenAPI o especificación de endpoints.
6. Matriz de roles y campos autorizados.
7. Dataset sintético/no sensible para desarrollo.
8. Runbook de operación, recuperación y escalamiento.

18. Cierre

La recomendación central es conservar la separación de responsabilidades: las fuentes y el pipeline calculan los datos operativos; el formulario captura el contexto humano; B-maker controla el acceso; y Next.js presenta una vista simple, trazable y accionable. Esta arquitectura permite evolucionar el MVP sin volver a diseñar la lógica del handoff.

Contacto funcional sugerido: responsable del proceso Handoff SMB. Validadores requeridos: Data/BI, Automation, Seguridad/Privacidad y líder de Customer Success/KAM.